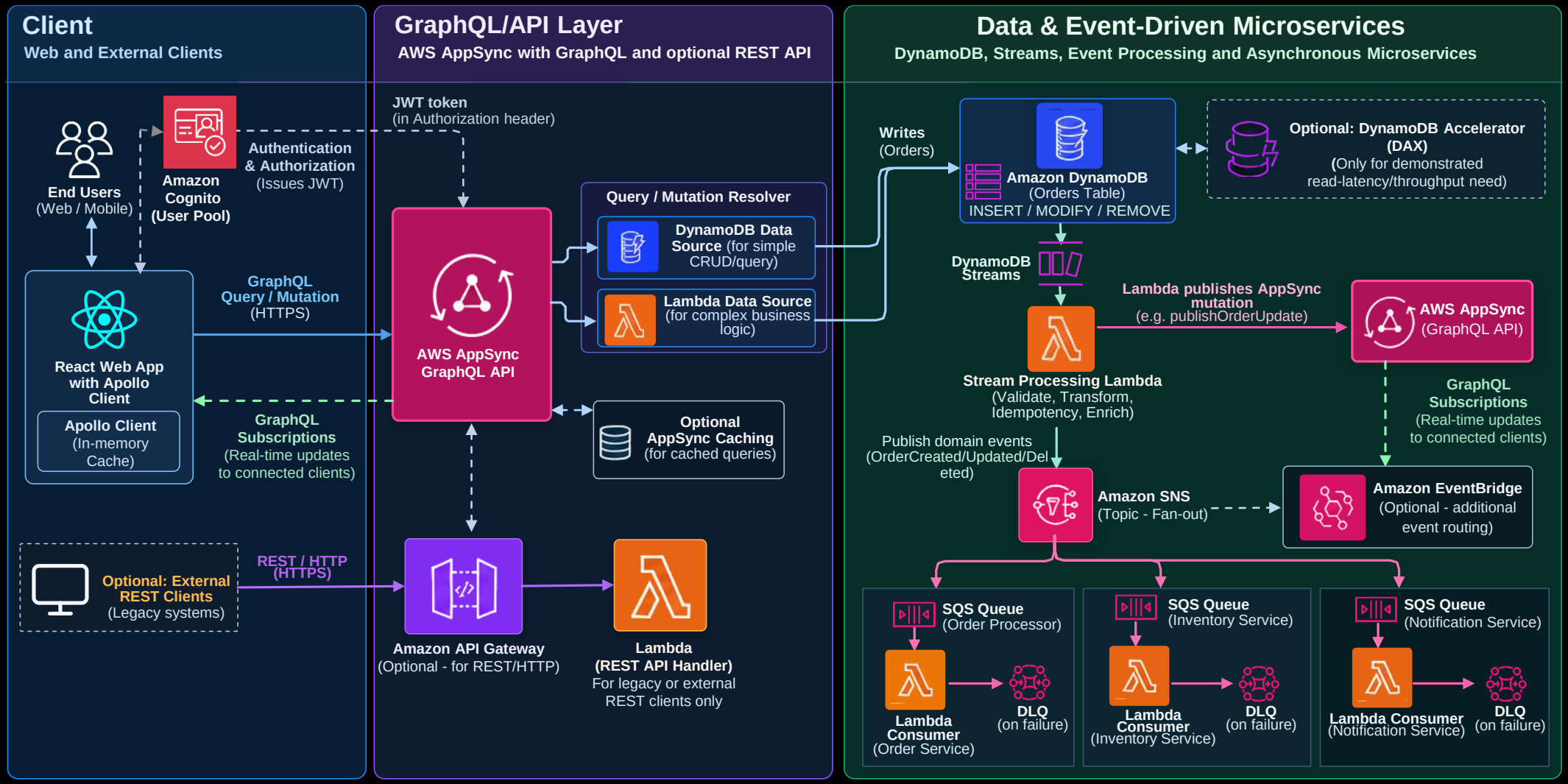
Max Montaseri

Back-End & Front-End Integration

AWS Serverless Microservices

aws

AWS Serverless Microservices: AppSync GraphQL + DynamoDB Streams



Security & Observability (Cross-Cutting Concerns)

AWS IAM
Least-privilege roles for AppSync, Lambda, DynamoDB, SNS, SQS, etc.

AWS KMS
Encryption at rest (DynamoDB, SQS, etc.) and in transit (TLS)

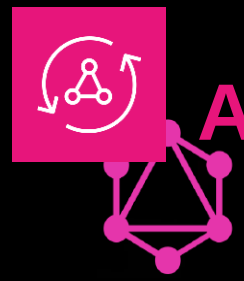
Amazon CloudWatch
Logs, Metrics, Alarms, Dashboards

AWS X-Ray
Distributed tracing (across AppSync, Lambda, etc.)

Reliability
Retry policies Idempotency
Dead-letter queues (DLQs)
Error handling

Legend

- Synchronous GraphQL equest/response (HTTPS)
- Real-time subscription flow (WebSockets)
- Asynchronous event flow (SNS/SQS/EventBridge)
- Optional flow



AWS AppSync GraphQL

vs

RESTful API





RESTful API (Representational State Transfer Application Programming Interface)

When a RESTful API is called, the server will transfer a representation of the requested resource's state to the client system

RESTful API

Uses HTTP verbs: POST, PUT, PATCH, DELETE, GET

- Insert: **POST**
- Update: **PUT**
- Partially update: **PATCH**
- Delete: **DELETE**
- Return all: **GET**
- Return one: **GET/Id**

Get all => URL/customer/
Get: => URL/customer/C100

RESTful API is entity based components

GET https://dell.ut.com/api/customers
GET https://dell.ut.com/api/customers/1

POST https://dell.ut.com/api/customers

PUT https://dell.ut.com/api/customers/1

DELETE https://dell.ut.com/api/customers/12

```
POST /echo/post/json HTTP/1.1
Host: reqbin.com
Accept: application/json
Content-Type: application/json
Content-Length: 81
{
     "Id": 78912,
     "Customer": "Jason Sweet",
     "Quantity": 1,
     "Price": 18.00
}
```

HTTP headers

Request headers are the metadata exchanged between the client and server. For instance, the request header indicates the format of the request and response, provides information about request status, and so on

Data (Jason body)

REST API requests might include data for the POST, PUT, and other HTTP methods to work successfully

Parameters (Jason body)

RESTful API requests can include parameters that give the server more details about what needs to be done. The following are some different types of parameters:

- Path parameters that specify URL details.
- Query parameters that request more information about the resource.
- Cookie parameters that authenticate clients quickly.

aws



GraphQL

GraphQL concept is implemented in different technology stacks by third parties such as AWS AppSynch

Not HTTP verbs any more! ~~**PUT, PATCH, DELETE, GET**~~

GraphQL Components

1. Query: Read data
pass variables for DB fetch/Get

2. Mutation: Write data
passing variables for DB Insert/Update/Delete

3. Subscription:

- Like queries, subscriptions enable you to fetch data
- *Unlike* queries, subscriptions are long-lasting operations that can change their result over time
- They can maintain an active connection to your GraphQL server (most commonly via WebSocket), enabling the server to push updates to the subscription's result
- Subscriptions are useful for notifying your client in real time about changes to back-end data, such as the creation of a new object or updates to an important field.

Put query commands in the request body

- Insert: **Mutation**
- Update: **Mutation**
- Partially update: **Mutation**
- Delete: **Mutation**
- Return all: **Query**
- Return one: **Query**

GraphQL Advantages

- Flexibility: Customizing what DB fields we're looking for
- Database Performance: Decrease database overload regarding technology stack such as AWS AppSync DynamoDB resolvers automatically apply the queries only on mentioned fields
- Network Performance: Decrease network overload with data fetch customization (OData for remediate RESTful API)
- GraphQL Resolver fields: Implementing aggregation functions if you have it in sent query

Exp. For phone devices

- Minimal data is important
  RESTful API returns all entity's fields (bulk data)
  Show up lists in cart view not in Grid (limited screen)
- Network overload in returned KBs
  Design different GraphQL endpoint for phone devices

aws

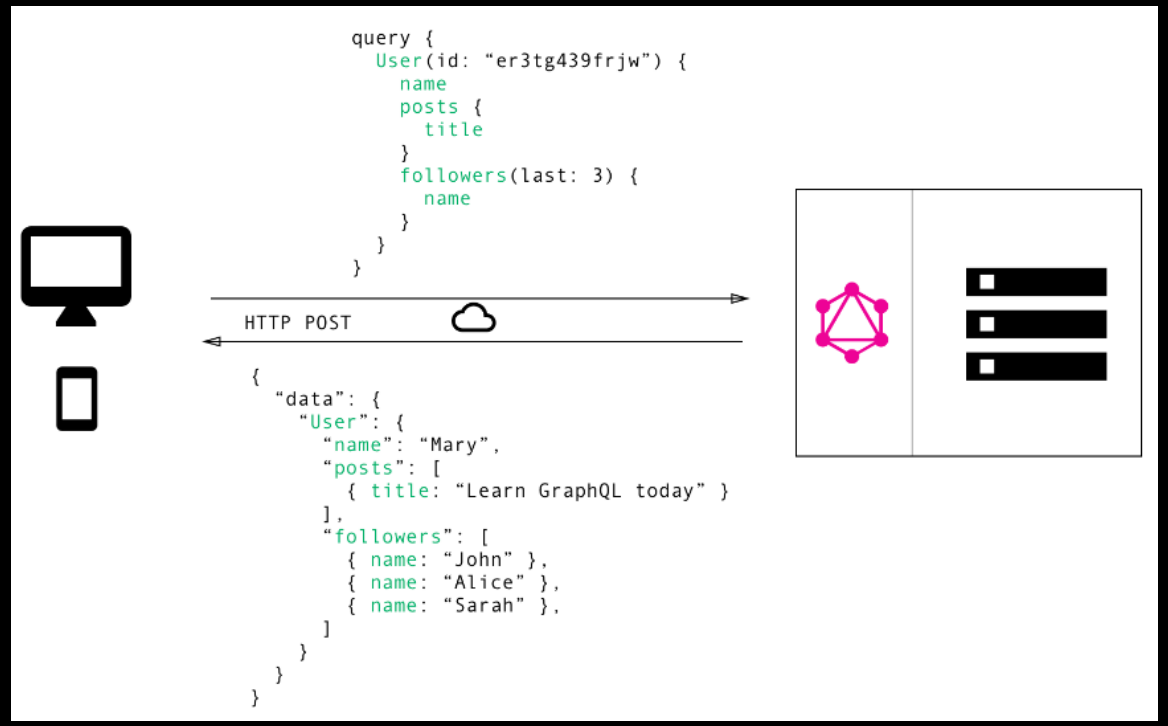
GraphQL

vs

RESTful API

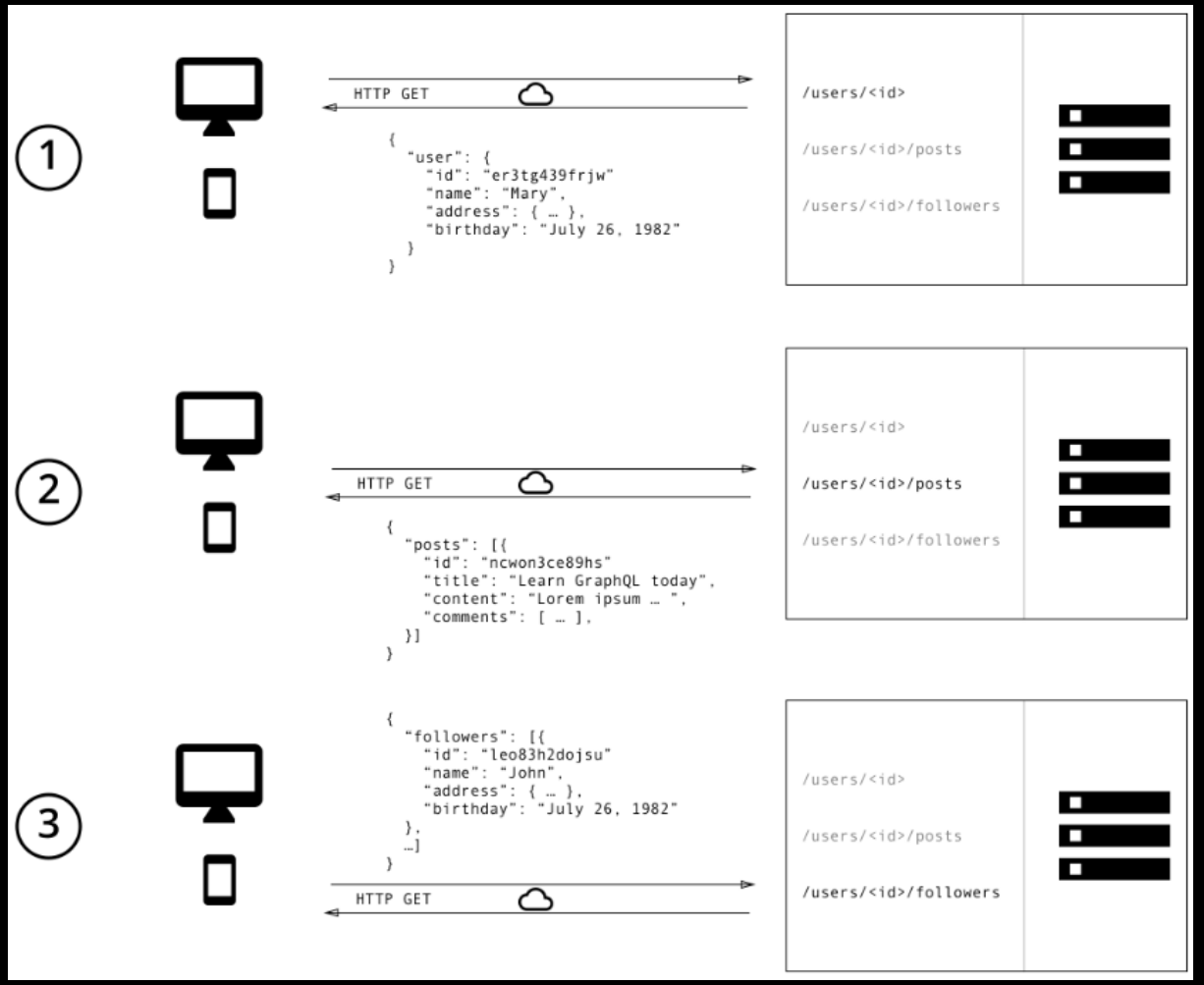
Data Fetching

In GraphQL on the other hand, you'd simply send a single query to the GraphQL server that includes the concrete data requirements. The server then responds with a JSON object where these requirements are fulfilled.



Data Fetching

With a REST API, you would typically gather the data by accessing multiple endpoints



aws



AWS AppSync

Implemented GraphQL technologically

✓ A single data API: Simplify application development with GraphQL APIs by providing a single endpoint to securely query or update data from multiple databases, microservices, and APIs.

✓ Built for real-time experiences: Easily create engaging real-time experiences with Pub/Sub APIs by automatically publishing data updates to subscribed API clients via serverless WebSockets connections.

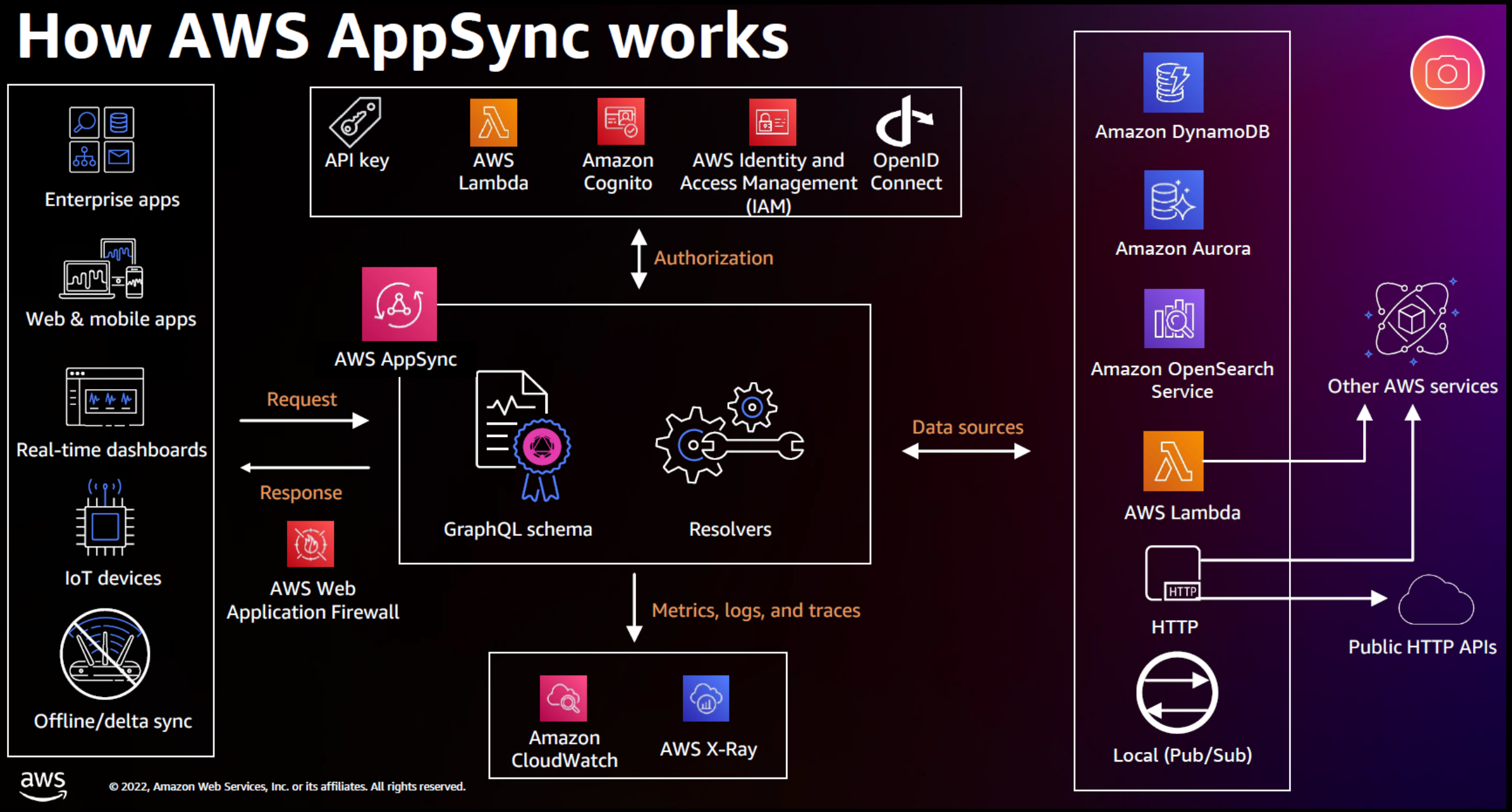
✓ 100% serverless: Secure, monitor, log, and trace your APIs via built-in support for AWS WAF, CloudWatch and X-Ray. Fully managed setup, administration, auto-scaling, and high-availability.



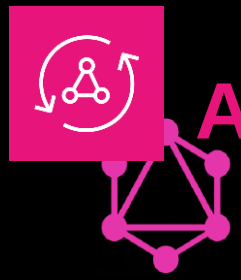
Node.js

Uses Apollo Server (server-side) and Apollo Client (client-side) for Implemented GraphQL concept

How AWS AppSync works



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.



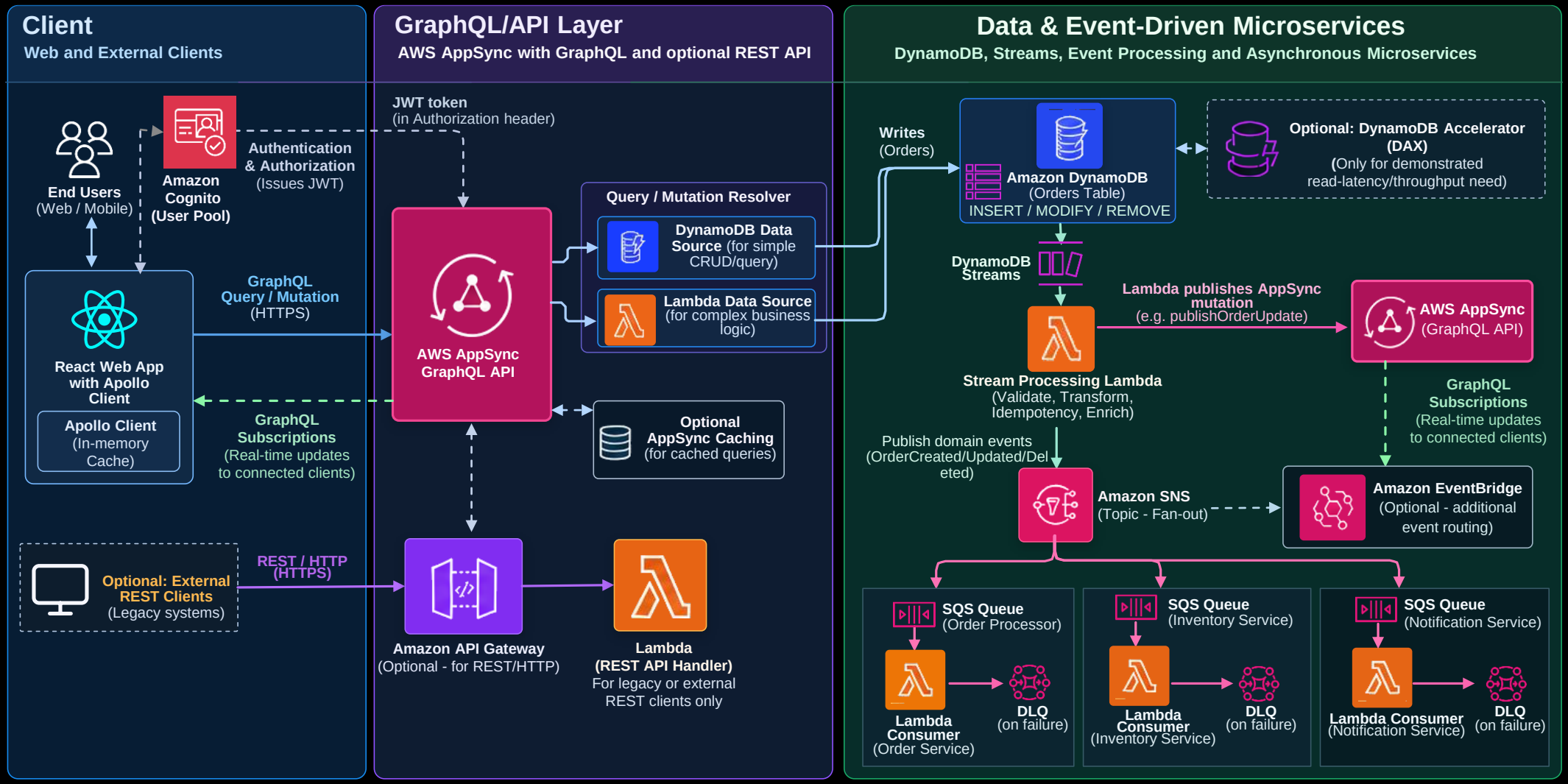
AWS AppSync GraphQL

AWS Serverless Architecture

Back-End & Front-End Integration

aws

AWS Serverless Microservices: AppSync GraphQL + DynamoDB Streams



Security & Observability (Cross-Cutting Concerns)

AWS IAM
Least-privilege roles for AppSync, Lambda, DynamoDB, SNS, SQS, etc.

AWS KMS
Encryption at rest (DynamoDB, SQS, etc.) and in transit (TLS)

Amazon CloudWatch
Logs, Metrics, Alarms, Dashboards

AWS X-Ray
Distributed tracing (across AppSync, Lambda, etc.)

Reliability
Retry policies Idempotency
Dead-letter queues (DLQs)
Error handling

Legend

- Synchronous GraphQL equest/response (HTTPS)
- Real-time subscription flow (WebSockets)
- Asynchronous event flow (SNS/SQS/EventBridge)
- Optional flow

Infrastructure Stack

The **Infrastructure Stack** provisions all common AWS services and Micrometrics only use them.

1. Networking (VPC)
2. DynamoDB
3. DynamoDBVpcEndpoint-prod Type: Gateway
4. VpcLambda-sg-prod Rule: allowAllOutbound
5. ApiVpcEndpoint-prod Type: Interface For execute API from VPC
6. VpcEndpoint-sg-prod Ingress rule: https or port 443 and allowAllOutbound rule
7. cloudfront-edge-apigateway
8. Custom Domain for invoking REST API: cloudfront-edge-apigateway.prod.domain.org
9. Amazon Route 53 Alias record: prod.domain.org Target: cloudfront-edge-apigateway.-prod.domain.org
10. Amazon CloudFront Distribution
11. AWS Certificate Manager (ACM)
12. AWS Key Management Service (KMS)

Microservice Back-End Stack

Hierarchy (Nested or Root-Child) AWS CloudFormaton Stack Because AWS has a limitation in resource numbers (Error)

Each Microservice has its **CDK Stack** and Microservices are Isolated in

1. Data (DynamoDB)
2. Logic (Lambda)

No data and logic redundancy!

The **Microservice CdkStack** provisions all private AWS services for each microservice

1. DynamoDB Table (fully independent): table properties (fields)
2. Lambda (fully independent): Don't call/invoke Lambda functions (Logical dependency) between Microservices => Lambda is triggered by APIGateway or SQS new message
3. API Gateway (fully independent): Lambda API methods (POST, GET, PUT, PATCH)

4. SQS (not fully independent): Pushes message to each subscribed Microservices for using the item
5. SNS (not fully independent): Notifies subscribed Microservices after event triggered

6. Add Dependency: DynamoDbStack output (DynamoDB Table) => pass to => SNSSQSTopicStack props => pass to => LambdaStack props => pass to => APIGateWayStack

aws